

GPU Kernel Programming: Performance Guidelines and Patterns

Advanced GPU Short Course

Piotr Jarosik, Marcin Lewandowski, Piotr Karwat

us4us Ltd.

Agenda

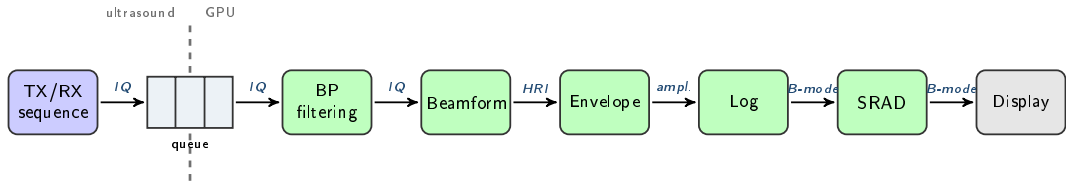
Performance Guidelines

- Memory access patterns
- GPU libraries
- Control flow
- Multiprocessor occupancy

HPC Patterns

- Stencil
- Iterative stencil
- Parallel reduction

Example: Ultrasound B-mode image reconstruction



Performance Guidelines

Performance guidelines

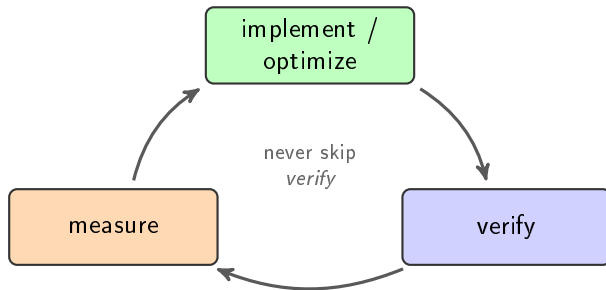
In this section we will consider the following aspects of optimizing CUDA kernels:

- **memory access patterns:** *memory coalescing* for the best throughput,
- **GPU libraries:** the fastest kernel is typically the one you do not write – what libraries are available and how to use them,
- **control flow:** how code branching can affect the performance,
- **multiprocessor occupancy:** experimenting with different block sizes.

You will also learn how profiling metrics can tell us where the performance issues are.

This section is mostly based on the content available in the *CUDA C++ Best Practices Guide*,
<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>

Iterative improvements



The kernel's performance can be evaluated using:

- **NVIDIA:** Nsight Systems (timeline, whole application), Nsight Compute (one kernel, metrics)
- **AMD:** rocprof (counters, traces), rocprof-sys (formerly Omnitrace, timeline), rocprof-compute (formerly Omnipperf, kernel analysis)

Performance metrics

what you suspect	NVIDIA – Nsight Compute (ncu)	AMD – rocprofv3 / rocprof-compute
memory access pattern	<code>smsp__sass_average_data_bytes_per_sector_mem_global_op_ld.pct</code> (and <code>_st</code>); sectors / request: <code>litex__t_sectors_pipe_lsu_mem_global_op_ld.sum ÷ litex__t_requests_pipe_lsu_mem_global_op_ld.sum</code>	<code>FETCH_SIZE</code> , <code>WRITE_SIZE</code> , <code>TCP_TCC_READ_REQ_sum</code> , <code>MemUnitStalled</code>
cache behaviour	<code>litex__t_sector_hit_rate.pct</code> , <code>lts__t_sector_hit_rate.pct</code>	<code>TCC_HIT_sum</code> , <code>TCC_MISS_sum</code> ; vL1D / L2 hit rate (rocprof-compute)
divergence	<code>smsp__thread_inst_executed_per_inst_executed_ratio</code> , <code>smsp__sass_average_branch_targets_threads_uniform.pct</code>	VALUUtilization (active lanes per wavefront)
occupancy	<code>sm__warps_active.avg.pct_of_peak_sustained_active</code> , <code>launch__registers_per_thread</code>	<code>MeanOccupancyPerCU</code> , <code>SQ_WAVES</code> ; wavefront occupancy (rocprof-compute)
throughput	<code>sm__throughput.avg.pct_of_peak_sustained_elapsed</code> , <code>dram__throughput.avg.pct_of_peak_sustained_elapsed</code>	<code>VALUBusy</code> , <code>TA_TA_BUSY_sum</code> ; roofline (rocprof-compute)

Profiler performance metrics: example output

Which kernel should we optimize?

First, measure the performance (Nsight Systems)

```
$ nsys profile -o fir ./fir_filter -n=4194304 -taps=64 -width=1024
$ nsys stats -report cuda_gpu_kern_sum fir.nsys-rep
```

Time (%)	Instances	Med (ns)	Name
61.0	56	59 668 587	convolve1d_axis0
19.0	106	9 617 528	convolve1d_global
9.0	56	9 575 561	convolve1d_axis1
6.0	56	5 823 357	convolve1d_shared
2.0	56	2 827 110	convolve1d_shared_const

Then verify why this one kernel:

```
$ nv-nsight-cu-cli -k convolve1d_axis0 -c 1 -metrics smp__inst_executed_global_loads_sum,\
    lts__request_tex_read_sectors_global_ld_uncached,smp__inst_executed_global_stores_sum,\
    lts__request_tex_write_sectors_global_nonatom ./fir_filter ... (same for axis1)
```

	load inst.	sectors read	ld sect./inst	store inst.	sectors written	st sect./inst
convolve1d_axis0	16 777 216	275 774 464	16.4	131 072	4 194 304	32.0
convolve1d_axis1	16 777 216	48 640 000	2.9	131 072	524 288	4.0

CUDA Best Practices Guide: memory coalescing

The concurrent accesses of the threads of a warp will coalesce into a number of transactions equal to the number of **32-byte transactions** necessary to service all of the threads of the warp.

Implement a GPU kernel that only loads *useful* data from global memory. We can achieve this with **coalesced memory accesses**:

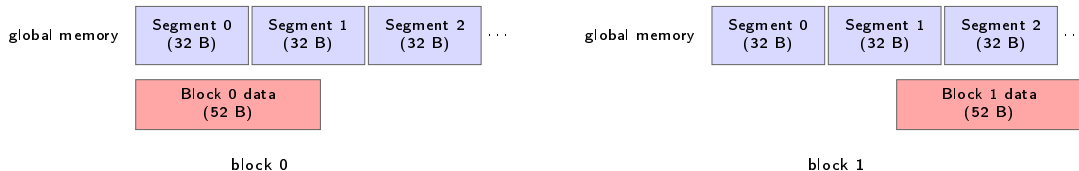
- the number of threads per block should be a **multiple of 32 threads**,
- sequential threads in a warp access memory that is sequential, with no offset.

Impact of misaligned accesses

CUDA Best Practices Guide

The number of threads per block should be a **multiple of 32 threads**, because this provides optimal computing efficiency and facilitates coalescing.

Typically, we assign one thread to read/write one output voxel/texel/sample/etc.



Impact of misaligned accesses: experiments

Is there any observable performance difference between `log_compression` launched with **256** and **261** threads in a block?

block size	kernel time	sector efficiency	L2 read sectors
256 (aligned)	0.411 ms	100.0 %	2.10 M
261 (misaligned)	0.459 ms	80.6 %	2.15 M (+2.6 %)

12 % slower – less than the 24 % extra sectors each warp moves, because adjacent warps **reuse the cached data** their neighbors fetched:

CUDA Best Practices Guide

So while the impact is still evident it is not as large as we might have expected. It would have been more so if adjacent warps had not exhibited such a high degree of reuse of the over-fetched cache lines.

Still, **setting a block size to a multiple of warp size** is a good rule of thumb.

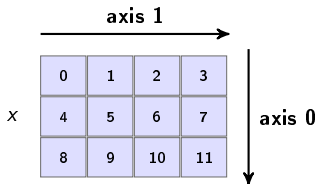
Impact of strided accesses

Guideline

Threads with successive identifiers read should the data from successive memory areas:

```
x = data[(some custom offset) + threadIdx.x]
```

Important when working with n-d array – the choice of the axis, along which a specific operation is performed, can affect effective bandwidth.



NumPy / C store arrays in **row-major** order:

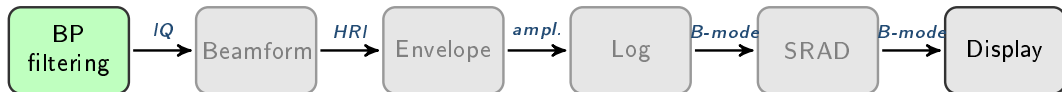
x	0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	---	----	----

MATLAB / Fortran store them **column-major**:

x	0	4	8	1	5	9	2	6	10	3	7	11
---	---	---	---	---	---	---	---	---	----	---	---	----

The same code walking “the fast axis” therefore means a *different* axis in the two conventions.

Example: Bandpass filter (convolution)



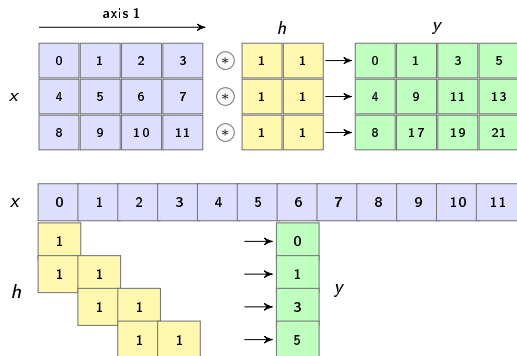
$$y[i] = \sum_{j=0}^{N-1} x[i + o - j] h[j]$$

Example: Bandpass filter (convolution)

```
// along axis 0
int sample = blockIdx.x*blockDim.x + threadIdx.x; // row
int scanline = blockIdx.y*blockDim.y + threadIdx.y; // column
for(int j = 0; j < n; ++j) {
    int k = sample + o - j;
    // TODO Simplified for clarity only. Remember to add guard here!
    value += x[k*width + scanline]*h[j];
}
y[sample*width + scanline] = value;
```

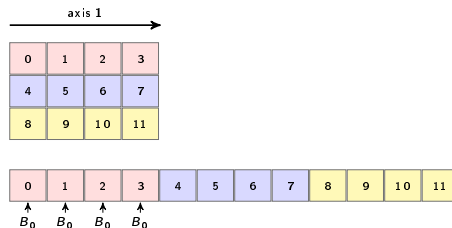
```
// along axis 1
int sample = blockIdx.x*blockDim.x + threadIdx.x; // column
int scanline = blockIdx.y*blockDim.y + threadIdx.y; // row
for(int j = 0; j < n; ++j) {
    int k = sample + o - j;
    // TODO simplified for clarity only. Remember to add guard here!
    value += x[scanline*width + k]*h[j]
}
y[scanline*width + sample] = value;
```

Example: Bandpass filter – Convolution along axis 1



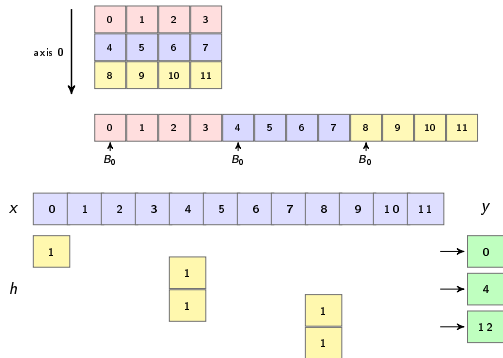
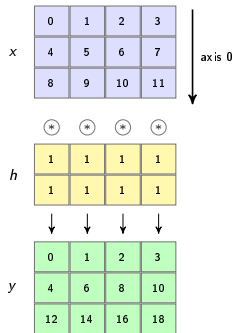
$y[0]$, $y[1]$, $y[2]$ and $y[3]$ are computed by threads with

`threadIdx.x` equal 0, 1, 2 and 3.



If the colors red, blue and yellow represent successive segments in the GPU memory, all readings for the computation of the first row fit in one block.

Example: Bandpass filter – Convolution along axis 0



$y[0]$, $y[1]$ and $y[2]$ are computed by threads with `threadIdx.x` equal 0, 1 and 2. The stride is expected to be much larger for the convolution along axis 0. Will it impact the bandwidth?

Impact of strided accesses: experiment results

33.5 M samples, 64 taps, 32768×1024 array.

kernel	gld/gst_eff. (any GPU)	time			
		MX250	RTX 3070	H100	MI300X
convolve1d_axis1	70 / 100 %	77.8 ms	5.38 ms	0.715 ms	0.759 ms
convolve1d_axis0	12.5 / 12.5 %	544.3 ms	71.50 ms	9.04 ms	28.35 ms
ratio		7.0×	13.3×	12.6×	37.3×

Example: `examples/03_fir_filter: --n=33554432 --taps=64 --width=1024, variants axis0 / axis1.`

GPU libraries: do not reinvent wheel

Guideline

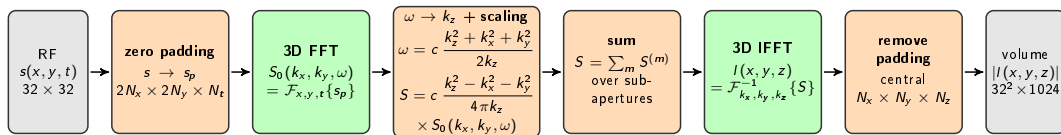
Before implementing a custom kernel, check whether there is a functionality ready to use in the vendor library.

A GPU library is usually optimized per architecture, and handles the edge cases you might have not thought of yet.

what you need	CUDA	HIP (AMD)
FFT	cuFFT	hipFFT / rocFFT
dense linear algebra	cuBLAS	hipBLAS / rocBLAS
sparse linear algebra	cuSPARSE	hipSPARSE
scan, reduce, sort, etc	CUB	rocPRIM
linear algebra solvers	cuSOLVER	hipSOLVER
image / signal primitives	NPP	rocAL, MIVisionX

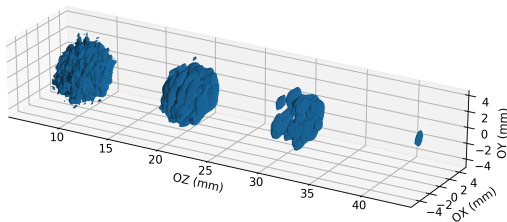
Example: 3D volume reconstruction (HSDI)

Hybrid Spectral-Domain Imaging (HSDI)



cuFFT / hipFFT – a library call

custom CUDA kernels



Why the frequency domain, for 3D

Time domain (delay-and-sum)

For every voxel, sum over every receiving element:

$$O\left(\underbrace{N_x N_y N_z}_{\text{voxels}} \times \underbrace{M^2}_{\text{channels}}\right)$$

A 32×32 aperture into a $32 \times 32 \times 1024$ volume is $1.0 \cdot 10^6$ voxels $\times$ 1024 channels $\approx 10^9$ accumulations – *per transmit*.

- The advantage grows with the aperture: doubling M costs the time domain $4\times$, the frequency domain almost nothing.
- This is what makes real-time 3D plausible on a single card.

Frequency domain (HSDI)

A few 3D transforms plus one pass over the spectrum:

$$O(N \log N), \quad N = 64 \cdot 64 \cdot 4096$$

The channel sum becomes a **superposition of spectra** – addition, not a loop. And the transforms are a library call.

How to use cuFFT: setup

```
#include <cuFFT.h>                                // link with -lcufft

cufftComplex *d_cube, *d_spec, *d_padded;          // interleaved complex (float2)
cudaMalloc(&d_cube,   nxz*nyz*nt*sizeof(cufftComplex)); // (2Nx, 2Ny, Nt)
cudaMalloc(&d_spec,   nxz*nyz*nz*sizeof(cufftComplex)); // (2Nx, 2Ny, Nz)
cudaMalloc(&d_padded, nxz*nyz*nz*sizeof(cufftComplex));

// one plan per transform size and type, created ONCE, reused every volume
cufftHandle plan_fwd, plan_inv;
cufftPlan3d(&plan_fwd, nxz, nyz, nt, CUFFT_C2C);    // 64 x 64 x 4096
cufftPlan3d(&plan_inv, nxz, nyz, nz, CUFFT_C2C);    // 64 x 64 x 1024
// ... reconstruct volumes ...
cufftDestroy(plan_fwd); cufftDestroy(plan_inv);
```

- **plan**: size, type (C2C, R2C, Z2Z for double) and batch
- layout: row-major, the **last** dimension is contiguous

How to use cuFFT: one volume

```
for (int em = 0; em < nem; ++em) {                                // RX sub-apertures
    zero_pad<<<grid_pad, block>>>(d_cube, d_rf + em*rf_size, ...);
    cufftExecC2C(plan_fwd, d_cube, d_cube, CUFFT_FORWARD); // in place
    interp_scale<<<grid_int, block>>>(d_spec, d_cube, ...,
                                      /*accumulate=*/em > 0); // + sum
}
cufftExecC2C(plan_inv, d_spec, d_padded, CUFFT_INVERSE); // out of place
// cuFFT does not normalize: scale by 1/(nxz*nyz*nz) while cropping
remove_padding<<<grid_crop, block>>>(d_volume, d_padded, ...,
                                     1.0f/(nxz*nyz*nz));
```

- **two library calls**, everything in between: padding, $\omega \rightarrow k_z$ interpolation + scaling, cropping
- `cufftExec*` is asynchronous, in the same way as a kernel launch

Control flow: how code branching affects the performance

Due to SIMT architecture of the CUDA multiprocessors, it is recommended to avoid different paths within the same warp.

CUDA Best Practices Guide

Flow control instructions (`if`, `switch`, `do`, `for`, `while`) can significantly affect the instruction throughput by causing threads of the same warp to diverge; that is, to follow different execution paths. If this happens, the different execution paths must be executed separately; this increases the total number of instructions executed for this warp.

Control flow: over-complicated example

```
// the easy way  
int r = i % 8;  
c[i] = r*a[i] + b[i];
```

```
// the overcomplicated way  
if(i % 8 == 0) {  
    c[i] = b[i];  
}  
else if(i % 8 == 1) {  
    c[i] = 1*a[i] + b[i];  
}  
// ...  
else if(i % 8 == 7) {  
    c[i] = 7*a[i] + b[i];  
}
```

kernel	time	branch_efficiency
simple	50.0 μ s	100.00%
overcomplicated	112.1 μ s	58.82%

branch_efficiency: ratio of non-divergent branches to total branches expressed as percentage.

Of course, the above example has been artificially complicated just to show the effect of complex kernel logic on the kernel's performance.

Control flow: in our example

`threadIdx.x` runs along `z`: a warp is 32 neighbouring depths of one line, and the only branch in the inner loop is a depth test.

```
for(int iRx = 0; iRx < nRx; ++iRx) {  
    // time of flight: the same code for every lane  
    if(iSamp < 0.0f || iSamp >= nSamp-1) { continue; } // outside the recording  
    // accumulate  
}
```

	branch_eff.	warp_exec_eff.
recording covers the image (--nsamp=1280)	100.00%	100.00%
image deeper than it (--nsamp=500)	100.00%	94.29%

τ grows monotonically with `z`, so the guard splits a warp at **one** boundary – and only where the image outruns the recording.

Multiprocessor occupancy: thread block size

Recall that: *the number of threads per block should be a multiple of 32 threads*, because this provides optimal computing efficiency and facilitates coalescing.

CUDA Best Practices Guide: rules of thumb

1. Threads per block should be **a multiple of warp size** to avoid wasting computation on under-populated warps and to facilitate coalescing.
2. A **minimum of 64 threads** per block should be used, and only if there are multiple concurrent blocks per multiprocessor.
3. Between **128 and 256 threads** per block is a good initial range for experimentation with different block sizes.
4. Use several smaller thread blocks rather than one large thread block per multiprocessor if latency affects performance. This is particularly beneficial to kernels that frequently call `__syncthreads()`.

Multiprocessor occupancy: experiments

achieved_occupancy: ratio of the average active warps per active cycle to the maximum number of warps supported on a multiprocessor.

block size	add_vectors time	achieved_occupancy
16	126.4 μ s	0.182
256	50.0 μ s	0.874
1024	49.3 μ s	0.813

Finding the right number of threads per block requires some experimentation, but generally **128 or 256 threads is a good starting point.**

Occupancy: in our example

The beamforming kernel, block size swept:

block	time	achieved_occupancy
8×8 (64)	22.50 ms	0.738
16×16 (256)	21.59 ms	0.737
32×8 (256)	23.46 ms	0.735
64×4 (256)	23.55 ms	0.735

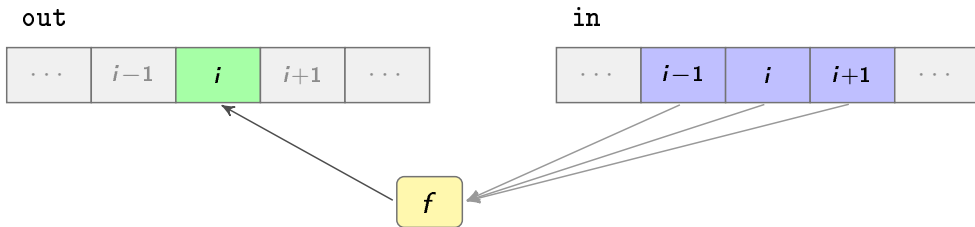
- Every candidate is already a multiple of 32: occupancy is **flat** to three decimals.
- The limit is elsewhere: **34 registers per thread** (`nvcc -Xptxas -v`) and the latency of the scattered sample reads.

HPC Patterns

Stencil

Task

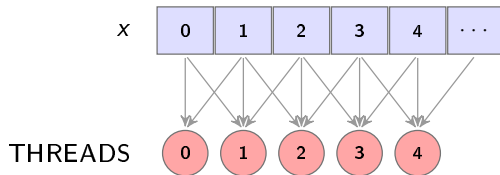
$$\text{out}[i] = f(\text{neighbourhood of in}[i])$$



- Examples: 1D/2D convolution: FIR filter (RF/IQ), smoothing, etc.
- Neighbouring threads read **overlapping** input windows
- Technique: **memory tiling**

Stencil: 1D convolution

Let's layout calculations for the 1D conv.

$$\begin{aligned}y[0] &= h[0]*x[1]+h[1]*x[0] \\y[1] &= h[0]*x[2]+h[1]*x[1]+h[2]*x[0] \\y[2] &= h[0]*x[3]+h[1]*x[2]+h[2]*x[1] \\y[3] &= h[0]*x[4]+h[1]*x[3]+h[2]*x[2] \\y[4] &= h[1]*x[4]+h[2]*x[3]\end{aligned}$$


Multiple threads read the same data from the global memory many times:

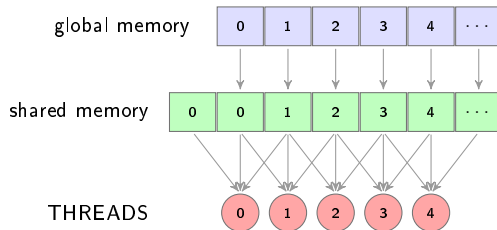
- reads $x[0]$ and $x[4]$ from the global memory twice,
- reads $x[1]$, $x[2]$ and $x[3]$ three times.

Stencil: the shared memory pattern

A typical CUDA shared memory usage pattern is as follows.

Each thread i in a block:

1. reads data from the global to shared memory,
2. waits until all other threads in the block finish 1.,
3. perform calculations using data available in the shared memory.



Stencil: 1D convolution

We can apply the above pattern to the `convolve_gpu_kernel` implementation:

- let's declare that each block has shared memory for $B + N - 1$ samples, where B is the size of the threads block, N is the length of the filter,
- for each thread i in a block:
 1. read a portion of data from global to shared memory,
 - start with $k = i + o - (N - 1)$,
 - read from global to shared memory: $x[k]$, $x[k + B]$, $\dots$, and so on, until you hit the end of the shared memory area,
 - if $k + jB < 0$ or $k + jB \geq m$ for any j , write 0 to shared memory at position $i + jB$, otherwise write $x[k + jB]$,
 2. wait until all threads in the block finish 1.,
 3. compute $y[i]$.

Stencil: step 1, read from global to shared memory

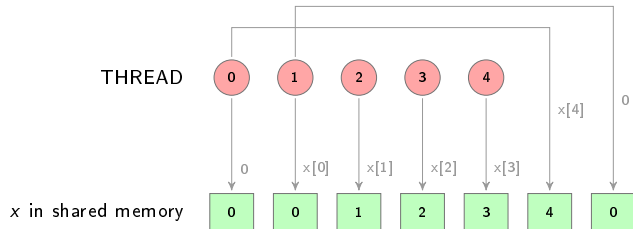
Our toy example, assume $B = 5$ (one block computes all the output samples), $N = 3$, $o = 1$:

$x = [0, 1, 2, 3, 4]$

$h = [0, 1, 2]$

$\text{shared_memory} = [0, 0, 0, 0, 0, 0, 0]$

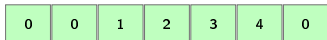
(note: an uninitialized shared memory may contain any arbitrary values, we used zeros just for clarity).



Stencil: steps 2 and 3

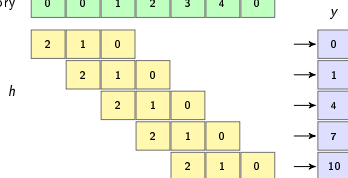
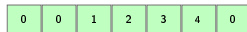
Step 2. Wait until threads finish the above procedure. At this point:

x in shared memory



Step 3. Compute y:

x in shared memory



Each $x[i]$ was read from the global memory exactly once.

Stencil: tile + halo

```
#define R 8
#define BLOCK 256
__constant__ float h[2*R + 1];

__global__ void fir_tiled(float *y, const float *x, int n) {
    __shared__ float tile[BLOCK + 2*R];
    int i = blockIdx.x*BLOCK + threadIdx.x;
    int t = threadIdx.x + R;
    tile[t] = (i < n) ? x[i] : 0.0f;           // body
    if (threadIdx.x < R) {                    // halos
        int l = i - R, r = i + BLOCK;
        tile[t - R] = (l >= 0) ? x[l] : 0.0f;
        tile[t + BLOCK] = (r < n) ? x[r] : 0.0f;
    }
    __syncthreads();                         // no early return before this line
    if (i >= n) return;
    float acc = 0.0f;
    #pragma unroll
    for (int k = -R; k <= R; ++k) acc += h[k + R]*tile[t + k];
    y[i] = acc;
}
```

- global reads per block: $B(2R + 1) \rightarrow B + 2R$

Stencil: results

1D convolution: $n = 2^{25}$ samples, 64 taps, block 256.

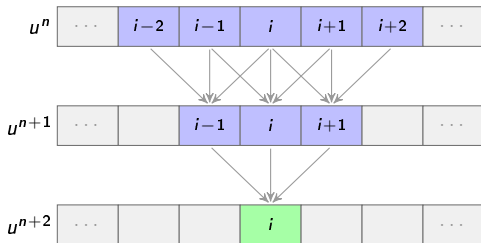
variant	NV MX250		NV 3070 Lap.		NV H100		AMD MI300X	
	time	sp.	time	sp.	time	sp.	time	sp.
global memory	77.5 ms	1.00×	5.29 ms	1.00×	0.648 ms	1.00×	0.726 ms	1.00×
+ tiling (shared)	46.5 ms	1.67×	4.58 ms	1.15×	0.589 ms	1.10×	0.408 ms	1.78×
+ filter in constant	21.3 ms	3.64 ×	2.51 ms	2.11 ×	0.334 ms	1.94 ×	0.384 ms	1.89 ×

Measured with `examples/03_fir_filter: ./build/fir_filter_cuda --n=33554432 --taps=64`, variants `global`, `shared`, `shared_const` (HIP: `fir_filter_hip`). Median of 30 iterations.

Iterative stencil

Task

$$u^{n+1}[i] = f(\text{neighbourhood of } u^n[i]), \quad \text{for every iteration } n = 0, 1, \dots, N_{it} - 1$$



- stencil, repeated N_{it} times
- iteration $n + 1$ needs **all** of iteration n to be done
→ **grid-wide barrier**, that is a kernel boundary
- ultrasound: anisotropic diffusion (Perona–Malik, **SRAD**), finite-difference wave simulation

Iterative stencil: the host loop

```
for (int it = 0; it < n_iter; ++it) {  
    diffusion_step<<<grid, block, 0, s>>>(v, u, w, h, lambda);  
    std::swap(u, v);           // ping-pong: never update in place  
}
```

- kernels in a single stream run in order: end of kernel = barrier for the next kernel invocation
- **double buffering** (ping-pong): otherwise in place → neighbours see old + new values

Example: SRAD

Speckle reducing anisotropic diffusion: Perona–Malik anisotropic diffusion, with the edge detector replaced by one that understands speckle.

Perona–Malik

$$\frac{\partial J}{\partial t} = \operatorname{div} (c \nabla J)$$

- $c \rightarrow 1$: smooth
- $c \rightarrow 0$: do not touch

Our objective is to diffuse *along* edges, not *across* them. The diffusion coefficient is what decides what an edge is.

The diffusion coefficient

$$c = \left[1 + \frac{q^2 - q_0^2}{q_0^2 (1 + q_0^2)} \right]^{-1}$$

- q – area brightness variance, *relative to its own brightness*
- q_0 – what pure speckle alone would give, measured on a homogeneous region
- $q \approx q_0$: just speckle $\Rightarrow$ homogeneous area, smooth it
- $q \gg q_0$: non-homogeneous, leave unchanged

One iteration, three kernels: (1) ROI statistics (reduction), (2) coefficient c (stencil), (3) update J (stencil).

Example: SRAD – the code

```
__global__ void srad_coef(float *c, const float *J, const float *q0sqr,
                        int nZ, int nX) {
    int z = blockIdx.x*blockDim.x + threadIdx.x;    // z (depth): fastest axis
    int x = blockIdx.y*blockDim.y + threadIdx.y;
    if (z >= nZ || x >= nX) return;
    float Jc = J[z + x*nZ];                        // the 4-neighbourhood,
    float dN = J[max(z-1, 0) + x*nZ] - Jc;          // clamped at the border
    float dS = J[min(z+1, nZ-1) + x*nZ] - Jc;
    float dW = J[z + max(x-1, 0)*nZ] - Jc;
    float dE = J[z + min(x+1, nX-1)*nZ] - Jc;
    c[z + x*nZ] = sradCof(Jc, dN, dS, dW, dE, *q0sqr); // the formula above
}

__global__ void srad_update(float *Jout, const float *J, const float *c,
                          int nZ, int nX, float lambda) {
    /* ... the same four differences of J ... */
    float cC = c[z + x*nZ], cS = c[zS + x*nZ], cE = c[z + xE*nZ];
    Jout[z + x*nZ] = Jc + 0.25f*lambda*(cC*dN + cS*dS + cC*dW + cE*dE);
}

for (int it = 0; it < n_iter; ++it) {                // the host loop
    srad_roi_stats<<<1, 256>>>(d_q0sqr, J, nZ, roi);    // reduction
    srad_coef    <<<grid, block>>>(d_c, J, d_q0sqr, nZ, nX); // stencil
    srad_update  <<<grid, block>>>(Jout, J, d_c, nZ, nX, lambda);
    std::swap(J, Jout);                                // ping-pong
}
```

Example: SRAD – results

512 × 512 pixels, 100 SRAD iterations, block 32 × 8.

variant	NV MX250		NV RTX 3070		NV H100		AMD MI300X	
	time	sp.	time	sp.	time	sp.	time	sp.
global, 3 kernels / iter.	15.1 ms	1.00×	2.02 ms	1.00×	1.43 ms	1.00×	1.86 ms	1.00×
+ 2D tile + halo	25.7 ms	0.59×	3.01 ms	0.67×	1.68 ms	0.85×	2.25 ms	0.83×
+ coef. & update fused	23.7 ms	0.64×	2.82 ms	0.72×	1.56 ms	0.91×	1.81 ms	1.03×
+ CUDA / HIP graph	23.6 ms	0.64×	2.66 ms	0.76×	1.36 ms	1.05×	1.80 ms	1.03×
baseline + graph, no tiling	15.1 ms	1.00×	1.75 ms	1.16×	1.11 ms	1.29×	1.80 ms	1.03×

Measured with examples/06_srad: ./build/srad_cuda --nz=512 --nx=512 --niter=100.

Parallel reduction

Task

$\text{out} = \text{in}[0] \oplus \text{in}[1] \oplus \dots \oplus \text{in}[n - 1], \quad \oplus \text{ associative: } +, \max, \min, \dots$

In beamforming:

$$g(t, x, z) = \sum_{r=1}^{N_r} f(t, r, \delta(t, r, x, z))$$

LRI: delay (interpolation) and **sum** over receivers

$$h(x, z) = \frac{1}{N_t} \sum_{t=1}^{N_t} g(t, x, z)$$

compounding: **average** of the LRIs

- also: the max for normalization / log-compression, the signal energy, the ROI statistics in SRAD
- our example: the **sum** of an array

Parallel reduction: sequential sum

```
__global__ void sum_sequential(float *out, const float *in, int n) {  
    float result = 0.0f;  
    for (int e = 0; e < n; ++e) {  
        result += in[e];  
    }  
    out[0] = result;  
}
```

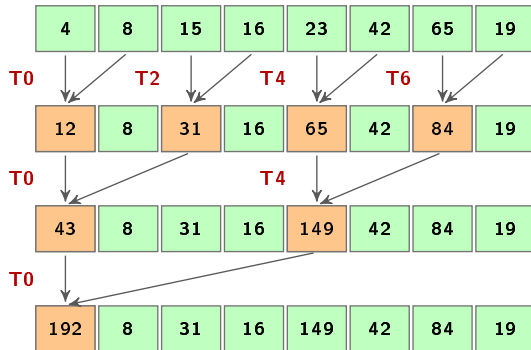
- $O(n)$ – optimal for a single thread
- but a single thread is used, and a GPU has thousands

Parallel reduction: the reduction tree

$O(\log n)$ steps, **shared memory**:

- threads with id divisible by 2:
 $h[id] += h[id + 1]$,
- then by 4: $h[id] += h[id + 2]$, ...

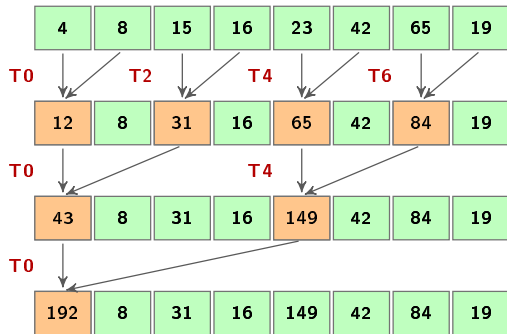
```
extern __shared__ float shared_memory[];
int thread = threadIdx.x;
int element = blockIdx.x*blockDim.x + thread;
shared_memory[thread] = in[element];
__syncthreads();
for (int s = 1; s < blockDim.x; s *= 2) {
    if (thread % (2*s) == 0) {
        shared_memory[thread] += shared_memory[thread + s];
    }
    __syncthreads();
}
if (thread == 0) {
    out[blockIdx.x] = shared_memory[0]; // one sum per block
}
```



more than one block: reduce the per-block sums again

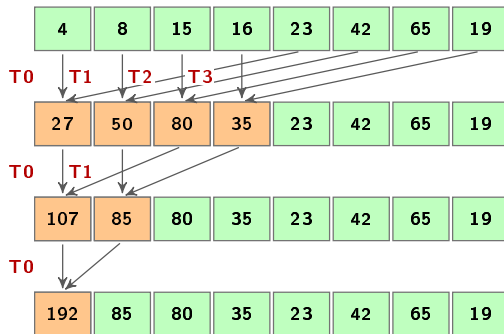
Parallel reduction: branch divergence

`thread % (2*s) == 0`: every second thread of a warp idles



```
for (int s = 1; s < blockDim.x; s *= 2) {
    if (thread % (2*s) == 0) { ... }
```

`thread < s`: the active threads are contiguous



```
for (int s = blockDim.x/2; s > 0; s >>= 1) {
    if (thread < s) {
        shared_memory[thread] += shared_memory[thread + s];
    }
    __syncthreads();
}
```

Parallel reduction: first add and warp shuffle

In the first step half of the threads idle → do the first addition **while loading** (half the blocks):

```
int element = blockIdx.x*(2*blockDim.x) + thread;
shared_memory[thread] = in[element] + in[element + blockDim.x];
```

The last 32 values: one warp, **registers only** – no shared memory, no `__syncthreads()`:

```
for (int s = blockDim.x/2; s >= 32; s >>= 1) { ... } // tree, down to one warp
if (thread < 32) {
    float v = shared_memory[thread];
    for (int offset = 16; offset > 0; offset >>= 1) {
        v += __shfl_down_sync(0xffffffff, v, offset); // lane i += lane i+offset
    }
    if (thread == 0) out[blockIdx.x] = v;
}
```

- in practice: `cub::DeviceReduce::Sum` (hipCUB, rocPRIM on AMD)

Parallel reduction: results

variant	$n = 1024$, block 1024		$n = 2^{24}$, block 256		
	time	speedup	time	GB/s	speedup
sequential (one thread)	12.8 μ s	1.0 $\times$	330 ms	0.2	1 $\times$
tree, interleaved (divergent)	4.8 μ s	2.6 $\times$	11.3 ms	5.9	29 $\times$
tree, sequential addressing	2.9 μ s	4.4 $\times$	5.19 ms	12.9	64 $\times$
+ first add during load	2.9 μ s	4.4 $\times$	2.63 ms	25.5	125 $\times$
+ warp shuffle	2.2 μ s	5.8 $\times$	1.69 ms	39.7	195 $\times$
<code>cub::DeviceReduce::Sum</code>	1.7 μ s	7.4 $\times$	1.31 ms	51.3	252$\times$

NVIDIA GeForce MX250, peak 56.1 GB/s. $n = 1024$: kernel time (Nsight Systems); $n = 2^{24}$: median of 30 iterations. Measured with `examples/08_reduction: ./build/reduction_cuda --n=16777216 --block=256`.

References

- *CUDA Best Practices Guide*, <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html>
- *Efficient Matrix Transpose in CUDA C/C++* (NVIDIA Devblog Post)
- *How to Access Global Memory Efficiently in CUDA C/C++ Kernels* (NVIDIA Devblog Post)
- M. Lewandowski, P. Jarosik, Y. Tasinkevych, M. Walczak, *Efficient GPU Implementation of 3D Spectral Domain Synthetic Aperture Imaging* – the HSDI pipeline of the last example (examples/04_hsdi_volume)
- *cuFFT / hipFFT* documentation, <https://docs.nvidia.com/cuda/cufft/>